```
    case 2:
      for (int i=0; i<2; i++) {
        digitalWrite(ledPin, HIGH);
        delay(interval/2);
        digitalWrite(ledPin, LOW);
        delay(interval/2);
      }
      break;
    case 3:
      for (int i=0; i<3; i++) {
        digitalWrite(ledPin, HIGH);
        delay(interval/3);
        digitalWrite(ledPin, LOW);
        delay(interval/3);
      }
      break;
    case 4:
      for (int i=0; i<4; i++) {
        digitalWrite(ledPin, HIGH);
        delay(interval/4);
        digitalWrite(ledPin, LOW);
        delay(interval/4);
      }
    }
    buttonCounter = 0;
  }
  delay(20);
}
```

Admittedly, we are jumping ahead a little bit using some time functions that we will not cover in detail until Chapter 7, but this is a pretty neat example of creating modes of behavior based on a range of input conditions. In this case, we can push the button up to four times in a half-second interval and the LED will blink a number of times equal to the number of button presses. No button presses, or button presses that exceed four, will be ignored—not exactly the best case for interaction design but it works for our example.

While still fundamentally building off of the last two examples, the challenge with this type of problem is that the button presses need to happen within a certain time frame or the Arduino will sit there counting button presses forever. To make this work, we have added two unsigned long variables, `startTime` and `interval`, and are using the `millis()` function, explained in more detail later, that will provide us with a time stamp when the first button is pressed. Adding a known interval to this initial time stamp will allow us to determine if a sufficient interval has passed, in this case a half second.

Once this interval has been passed, the code proceeds to the `switch` statement that compares the variable `buttonCounter` to four different cases. Each case corresponds to how many times we want to blink, beginning with 1 blink in case 1 and working its way up to 4 blinks in case 4. After the LED has been blinked, we use the `break` statement to exit the `switch` statement to prevent unwanted blinks. In this example, we decided to keep the total number of blinks all to within 1 second by using the `interval` variable in the delay, dividing it by the number of times we want it to blink.

Moving on from here, we could continue to add cases to the switch statement and make a game out of how many times the button can be pressed in the interval time. We could also provide some sort of feedback when we don't receive the input that we want to receive, letting the user know that they should try again.